

## Лабораторная работа №5. Асинхронность в Dart и Flutter.

### Создание приложения «Фото дня»

**Цель работы:** Понять концепцию асинхронного программирования — изучить **Future, async/await** в **Dart**. Применить эти знания во **Flutter**-приложении, которое загружает случайные фотографии из интернета.

#### Необходимые инструменты:

- Flutter 3.0+
- VS Code с расширением Flutter
- Git
- Браузер Google Chrome

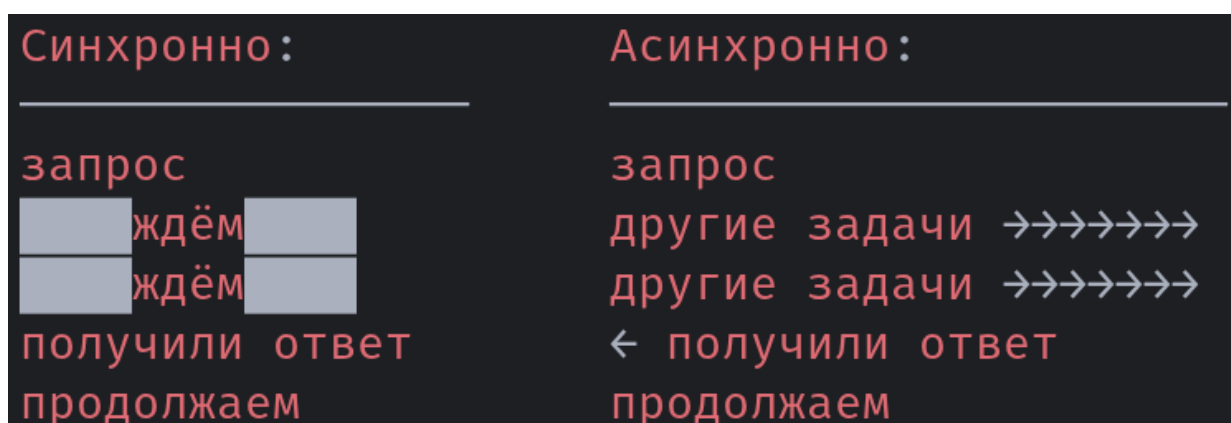
#### Теоретическая часть. Зачем нужна асинхронность?

Представьте: вы заказали пиццу по телефону. Есть два варианта поведения:

**Синхронный** — вы кладёте трубку, садитесь у двери и ждёте. Вы не можете ничего делать, пока пицца не придет. Всё заморожено.

**Асинхронный** — вы кладёте трубку, занимаетесь своими делами. Когда курьер звонит в дверь — вы прерываетесь и забираете пиццу.

В программировании то же самое. Когда приложение делает запрос к серверу или читает файл — это занимает время. Если ждать **синхронно** — интерфейс зависнет, пользователь не сможет ничего делать. Если ждать **асинхронно** — приложение остаётся живым.



В **Dart** асинхронность встроена в язык через **Future** и **async/await**.

#### Часть 1: Async/Await в Dart — консольная практика

##### Шаг 1. Создание проекта

1. Перейдите в папку с лабораторными:

**cd Documents/Flutter/**

2. Создайте консольный проект:

```
dart create lab5_async
```

```
cd lab5_async
```

```
git init -b main
```

3. Откройте в VS Code:

```
code .
```

4. Удалите файлы `lib/lab5_async.dart` и `test/lab5_async_test.dart`. Откройте `bin/lab5_async.dart`, удалите содержимое.

## Шаг 2. Что такое Future?

`Future<T>` — это объект, который **обещает** вернуть значение типа `T` в будущем. Сейчас результата нет, но он появится позже.

Напишите в `bin/lab5_async.dart`:

```
Future<String> fetchMessage() {
  return Future.delayed(
    Duration(seconds: 2),
    () => 'Привет из будущего!',
  ); // Future.delayed
}

Run | Debug
void main() {
  print('Начало программы!');
  fetchMessage().then(
    (message) => print(message),
  );
  print('Конец main()');
}
```

Запустите:

```
dart run
```

Ожидаемый вывод:

```
└─$ dart run
Building package executable...
Built lab5_async:lab5_async.
Начало программы!
Конец main()
Привет из будущего!
```

Обратите внимание на порядок: Конец `main()` напечатался **раньше** чем сообщение из `fetchMessage()`. Функция `main()` не стала ждать — она продолжила выполнение, а результат Future пришёл позже через `.then()`.

Разберём код:

- `Future.delayed(duration, () => значение)` — создаёт Future, который завершится через указанное время и вернёт значение
- `Duration(seconds: 2)` — объект времени, 2 секунды
- `.then((result) => ...)` — что сделать когда Future завершится

### Шаг 3. Ключевые слова `async` и `await`

`.then()` работает, но код быстро становится трудночитаемым при нескольких последовательных операциях. `async/await` делают асинхронный код похожим на обычный синхронный.

Замените содержимое файла:

```
Future<String> fetchMessage() async {
  await Future.delayed(Duration(seconds: 2));
  return 'Привет из будущего!';
}

void main() async {
  print('Начало программы!');

  print('Загружаем сообщение...');
  String message = await fetchMessage();
  print(message);

  print('Конец программы');
}
```

Запустите и сравните вывод с предыдущим:

```
↳$ dart run
Building package executable...
Built lab5_async:lab5_async.
Начало программы!
Загружаем сообщение...
Привет из будущего!
Конец программы
```

Теперь порядок предсказуемый — **await** приостанавливает выполнение функции до получения результата.

Разберём:

- **async** после `()` — помечает функцию как асинхронную. Теперь внутри можно использовать `await`
- **await** перед вызовом — «подожди здесь пока Future не завершится, потом продолжи»
- **main()** тоже может быть **async** — это стандартная практика в Dart

**Важно:** `await` не блокирует весь поток — он только приостанавливает текущую функцию. Другие части программы продолжают работать.

#### Шаг 4. Последовательные запросы

В реальных приложениях часто нужно выполнить несколько асинхронных операций по очереди. Добавьте под предыдущим кодом:

```
Future<String> fetchUser() async {  
  await Future.delayed(Duration(seconds: 1));  
  return 'Пользователь: Артём';  
}
```

```
Future<String> fetchAvatar() async {  
  await Future.delayed(Duration(seconds: 1));  
  return 'Аватар: avatar.png';  
}
```

```
Future<void> loadProfile() async {  
  print('Загружаем профиль...');  
  
  String user = await fetchUser();  
  print(user);  
  
  String avatar = await fetchAvatar();  
  print(avatar);  
  
  print('Профиль загружен!');  
}
```

Вызовите `loadProfile()` из `main()`:

```
void main() async {
  await loadProfile();
}
```

Запустите. Операции выполняются строго последовательно — сначала пользователь, потом аватар. Общее время ~2 секунды.

**Future<void>** — функция асинхронная, но ничего не возвращает. **void** как обычно означает «нет возвращаемого значения».

### Шаг 5. Обработка ошибок

Сетевые запросы могут падать — сервер недоступен, нет интернета, неверный URL. Ошибки в асинхронном коде обрабатываются через **try/catch**:

```
Future<String> fetchData() async {
  await Future.delayed(Duration(seconds: 1));
  throw Exception(
    'Сервер недоступен!',
  ); // имитируем ошибку
}
```

```
void main() async {
  try {
    String data = await fetchData();
    print(data);
  } catch (e) {
    print('Ошибка: $e');
  }
}
```

### Часть 2: Flutter-приложение «Фото дня»

**Да, это второй проект внутри лабораторной работы. У вас будет два проекта.**

Теперь применим **async/await** в настоящем приложении. Будем загружать случайные фотографии собак и пейзажей через публичные API:

- **Dog API:** <https://dog.ceo/api/breeds/image/random> — возвращает JSON с URL случайной фотографии собаки
- **Landscape API:** [https://picsum.photos/seed/\\$random/800/800](https://picsum.photos/seed/$random/800/800) — возвращает JSON с массивом, первый элемент содержит URL фотографии пейзажа

## Шаг 1. Создание Flutter-проекта

1. Откройте терминал и перейдите в папку с лабораторными:

```
cd Documents/Flutter/
```

2. Создайте новый Flutter-проект:

```
flutter create photo_of_the_day
```

```
cd photo_of_the_day
```

```
code .
```

3. Создайте папку для скриншотов:

```
mkdir img
```

4. Настройте **.gitignore** — добавьте в конец файла:

```
android/
```

```
ios/
```

```
linux/
```

```
macos/
```

```
windows/
```

```
test/
```

```
.idea/
```

```
*.iml
```

```
.DS_Store
```

```
Thumbs.db
```

5. Сделайте коммит:

```
git add .
```

```
git commit -m "Initial Flutter project"
```

```
git push
```

6. Создайте **public** репозиторий на GitHub с названием **Flutter\_lab5**, привяжите и отправьте:

```
git remote add origin <URL_вашего_репозитория>
```

```
git push -u origin main
```



**СКРИНШОТ 1:** Скриншот терминала с выводом пуша. Сохраните в **img/step1\_ВашаФамилия.png**

## Шаг 2. Подключение пакета http

Для HTTP-запросов используем пакет **http** с pub.dev — официальный реестр пакетов Dart и Flutter (аналог PyPI для Python).

Откройте **pubspec.yaml**. Удалите ненужные комментарии. Далее найдите раздел **dependencies** и добавьте пакет:

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.6.0
  cupertino_icons: ^1.0.8
```

Сохраните файл. VS Code автоматически предложит установить зависимости — нажмите **Get Packages**. Либо выполните в терминале:

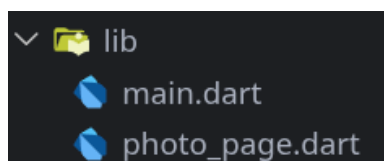
**flutter pub get**

### Шаг 3. Удаляем лишнее

Откройте **lib/main.dart**. Удалите **весь** код в файле и файл **test/widget\_test.dart**.

### Шаг 4. Структура файлов

Мы разобьём приложение на два файла:



- **lib/main.dart** — точка входа, MaterialApp
- **lib/photo\_page.dart** — главный экран с логикой загрузки фото

### Шаг 5. Пишем lib/photo\_page.dart

Создайте файл **lib/photo\_page.dart**

#### 5.1 Импорты

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
```

- **dart:convert** — стандартная библиотека для работы с JSON (jsonDecode)
- **package:http/http.dart as http** — пакет для HTTP-запросов. as http означает: обращаться к нему через префикс http.get(...), http.Response и т.д.

#### 5.2 Enum для выбора

Добавьте перечисление **вне класса**:

```
enum PhotoType { dog, landscapes }
```

**enum** — перечисление, набор именованных констант. Удобно когда переменная может принимать одно из нескольких фиксированных значений. Здесь: либо собака, либо пейзаж.

### 5.3 StatefulWidget

```
class PhotoPage extends StatefulWidget {  
  const PhotoPage({super.key});  
  
  @override  
  State<PhotoPage> createState() => _PhotoPageState();  
}
```

Вы уже знакомы с этой структурой — **StatefulWidget** для экрана, у которого меняется состояние.

### 5.4 State-класс — поля

```
class _PhotoPageState extends State<PhotoPage> {  
  String? _imageUrl;  
  bool _isLoading = false;  
  String? _errorMessage;  
  PhotoType _animalType = PhotoType.dog;  
}
```

- **String? \_imageUrl** - URL загруженной картинки (null = ещё не загружена)
- **bool \_isLoading = false** - идёт ли загрузка прямо сейчас
- **String? \_errorMessage** - сообщение об ошибке (null = ошибок нет)
- **PhotoType \_animalType = PhotoType.dog** - текущий выбор собаки

Три состояния экрана:

- **\_isLoading = true** — показываем индикатор загрузки
- **\_imageUrl != null** — показываем картинку
- **\_errorMessage != null** — показываем сообщение об ошибке

### 5.5 Метод загрузки фото

Пишем внутри класса **\_PhotoPageState**

```
20 Future<void> _fetchPhoto() async {  
21   setState(() {  
22     _isLoading = true;  
23     _errorMessage = null;  
24     _imageUrl = null;  
25   });  
26 }
```

продолжение на  
следующей странице...



```

27  try {
28      String url;
29      http.Response response;
30
31      if (_animalType == PhotoType.dog) {
32          url = 'https://dog.ceo/api/breeds/image/random';
33          response = await http.get(Uri.parse(url));
34          Map<String, dynamic> data = jsonDecode(
35              response.body,
36          );
37          _imageUrl = data['message'];
38      } else {
39          final random =
40              DateTime.now().millisecondsSinceEpoch;
41          _imageUrl = 'https://picsum.photos/seed/$random/800/800';
42      }
43      } catch (e) {
44          _errorMessage = 'Не удалось загрузить фото.\nПроверьте подключение к
интернету.';
45      }
46
47      setState(() {
48          _isLoading = false;
49      });
50  }

```

**Пока вы не допишете весь код класса `PhotoPageState` будут ошибки, это нормально!**

Разберём ключевые моменты:

`setState()` дважды — это намеренно:

- В начале: сразу показываем загрузку и сбрасываем предыдущий результат
- В конце: скрываем загрузку после получения результата

`http.get(Uri.parse(url))` — отправляет GET-запрос. `Uri.parse()` преобразует строку в объект `Uri` — Dart требует именно `Uri`, не строку. `await` ждёт ответа от сервера.

`jsonDecode(response.body)` — парсит JSON из тела ответа. Возвращает `Map` или `List` в зависимости от структуры JSON. Посмотрите что возвращают API в браузере:

- Dog API: `{"message": "https://images.dog.ceo/...", "status": "success"}`

Dog API возвращает **объект** → парсим как `Map`, берём `data['message']`.

## 5.6 Метод `build()` — переключатель фото

**Пишем внутри класса `_PhotoPageState`**

```

52  @override
53  Widget build(BuildContext context) {
54      return Scaffold(
55          appBar: AppBar(
56              title: const Text('Фото дня'),
57              centerTitle: true,
58          ), // AppBar
59          body: Column(
60              children: [
61                  const SizedBox(height: 20),
62                  _buildAnimalToggle(),
63                  const SizedBox(height: 20),
64                  _buildContent(),
65                  const SizedBox(height: 20),
66                  _buildButton(),
67                  const SizedBox(height: 20),
68              ],
69          ), // Column
70      ); // Scaffold
71  }

```

Мы разбиваем **build()** на несколько вспомогательных методов — каждый отвечает за свою часть UI. Это стандартная практика: когда дерево виджетов большое, читать один огромный **build()** тяжело.

## 5.7 Переключатель собака/пейзаж

**ChoiceChip** — виджет Material Design для выбора одного варианта из нескольких. **selected: \_animalType == AnimalType.dog** — чип подсвечен, если выбрана собака. При переключении сбрасываем предыдущую картинку.

Для дальнейшей вставки эмодзи - 🐶, 🌅

```

75   Widget _buildAnimalToggle() {
76       return Row(
77           mainAxisAlignment: MainAxisAlignment.center,
78           children: [
79               ChoiceChip(
80                   label: const Text('🐶 Собаки'),
81                   selected: _animalType == PhotoType.dog,
82                   onSelect: (selected) {
83                       if (selected) {
84                           setState(() {
85                               _animalType = PhotoType.dog;
86                               _imageUrl = null;
87                               _errorMessage = null;
88                           });
89                       }
90                   }, // ChoiceChip
91               const SizedBox(width: 12),
92               ChoiceChip(
93                   label: const Text('🌄 Пейзаж'),
94                   selected: _animalType == PhotoType.landscapes,
95                   onSelect: (selected) {
96                       if (selected) {
97                           setState(() {
98                               _animalType = PhotoType.landscapes;
99                               _imageUrl = null;
100                               _errorMessage = null;
101                           });
102                       }
103                   }, // ChoiceChip
104               ), // ChoiceChip
105           ],
106       ); // Row
107   }
108 }

```

## 5.8 Контент — три состояния экрана

Этот метод — сердце экрана. Он последовательно проверяет состояние и возвращает нужный виджет:

1. **\_isLoading == true** → крутилка **CircularProgressIndicator**
2. **\_errorMessage != null** → текст ошибки красным
3. **\_imageUrl != null** → картинка с скруглёнными углами
4. Иначе → подсказка «нажмите кнопку»

**`_imageUrl!`** — восклицательный знак означает «я уверен что это не null». Dart разрешает так делать после явной проверки **`if (_imageUrl != null)`**.

**`Expanded`** — виджет, который занимает всё доступное пространство в **`Column`**. Без него картинка не будет растягиваться.

**`ClipRRect`** — обрезает содержимое по скруглённым углам.

**`Image.network()`** — загружает и отображает картинку по URL. Это синхронный виджет — Flutter сам управляет загрузкой изображения внутри.

```
110   Widget _buildContent() {
111     if (_isLoading) {
112       return const Expanded(
113         |   child: Center(child: CircularProgressIndicator()),
114         |   ); // Expanded
115     }
116
117     if (_errorMessage != null) {
118       return Expanded(
119         |   child: Center(
120         |     |   child: Padding(
121         |     |     |   padding: const EdgeInsets.all(24),
122         |     |     |   child: Text(
123         |     |     |     |   _errorMessage!,
124         |     |     |     |   textAlign: TextAlign.center,
125         |     |     |     |   style: const TextStyle(
126         |     |     |     |     |   fontSize: 16,
127         |     |     |     |     |   color: Colors.red,
128         |     |     |     |     |   ), // TextStyle
129         |     |     |     |   ), // Text
130         |     |   ), // Padding
131         |   ), // Center
132       ); // Expanded
133     }
```

**продолжение на следующей странице...**

```

134
135     if (_imageUrl != null) {
136         return Expanded(
137             child: Padding(
138                 padding: const EdgeInsets.symmetric(
139                     horizontal: 16,
140                 ), // EdgeInsets.symmetric
141                 child: ClipRRect(
142                     borderRadius: BorderRadius.circular(16),
143                     child: Image.network(
144                         _imageUrl!,
145                         fit: BoxFit.cover,
146                         width: double.infinity,
147                     ), // Image.network
148                 ), // ClipRRect
149             ), // Padding
150         ); // Expanded
151     }
152
153     return const Expanded(
154         child: Center(
155             child: Text(
156                 'Нажмите кнопку\nчтобы загрузить фото',
157                 textAlign: TextAlign.center,
158                 style: TextStyle(
159                     fontSize: 18,
160                     color: Colors.grey,
161                 ), // TextStyle
162             ), // Text
163         ), // Center
164     ); // Expanded
165 }

```

## 5.9 Кнопка

**onPressed:** `_isLoading ? null : _fetchPhoto` — если идёт загрузка, кнопка отключается (`null` = нет обработчика = кнопка неактивна). Это стандартный паттерн в Flutter.

`_fetchPhoto` без скобок — мы передаём **ссылку на функцию**, а не вызываем её. `_fetchPhoto()` со скобками вызвало бы функцию немедленно при построении виджета.

```

166
167   Widget _buildButton() {
168       String label = _animalType == PhotoType.dog
169           ? '🐶 Новая собака'
170           : '🌅 Новый пейзаж';
171
172       return ElevatedButton(
173         onPressed: _isLoading ? null : _fetchPhoto,
174         child: Padding(
175           padding: const EdgeInsets.symmetric(
176             horizontal: 24,
177             vertical: 12,
178           ), // EdgeInsets.symmetric
179           child: Text(
180             label,
181             style: const TextStyle(fontSize: 16),
182           ), // Text
183         ), // Padding
184       ); // ElevatedButton
185   }
186 }

```

Теперь ошибки должны исчезнуть. Проверь файл на наличие ошибок, и внимательно перепроверьте код, в случае их обнаружения.

#### Шаг 6. Пишем lib/main.dart

```

import 'package:flutter/material.dart';
import 'photo_page.dart';

```

```

void main() {
  runApp(const MyApp());
}

```

```

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Фото дня',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.deepPurple,
        ), // ColorScheme.fromSeed
        useMaterial3: true,
      ), // ThemeData
      home: const PhotoPage(),
    ); // MaterialApp
  }
}

```

## Шаг 7. Запуск

**flutter run -d chrome**

Первый запуск займёт 1–2 минуты. После этого проверьте:

1. Нажмите кнопку «🐶 Новая собака» — появится крутилка, потом фото
2. Нажмите ещё раз — загрузится другое фото
3. Переключитесь на «🌄 Новый пейзаж» и нажмите кнопку снова

**Многие зарубежные сервисы (dog.ceo, picsum.photos, Unsplash) не будут работать в России без КВН из-за блокировок. Решение в шаге 9.**



**СКРИНШОТ 2:** Скриншот браузера с загруженным фото.

Сохраните в **img/step2\_ВашаФамилия.png**

Сделайте коммит:

**git add .**

**git commit -m "Add photo fetch with async/await"**

**git push**

## Шаг 8. Как это работает — полная цепочка

Разберём что происходит при нажатии кнопки:

1. onPressed вызывает `_fetchPhoto()`

2. `setState()` — `_isLoading = true`

Flutter перестраивает дерево → появляется `CircularProgressIndicator`

3. `await http.get(...)` — отправляем запрос к Dog/Cat API

Функция приостанавливается, Flutter продолжает работу (UI не заморожен)

4. Сервер возвращает JSON → `jsonDecode()` парсит его → получаем URL картинки

5. `setState()` — `_isLoading = false`, `_imageUrl =` полученный URL

Flutter перестраивает дерево → `CircularProgressIndicator` исчезает,  
появляется `Image.network()`

6. `Image.network()` внутри сам загружает картинку по URL и отображает её

Ключевой момент: между шагами 3 и 4 Flutter **не заморожен**. Пользователь видит крутилку, может крутить страницу — приложение живёт. Именно для этого нужен **async/await**.

## Шаг 9. Используем полностью локальные картинки

Без интернета посмотрим на чистую работу Future + индикатор загрузки.

**Внесём минимальные правки.**

1. Создайте папку в корне вашего проекта **assets/images/**

2. Положите в папку **images/** 3-6 любых картинок (названия используйте: **photo1.jpg**, **photo2.jpg** ...).

3. В **pubspec.yaml** добавьте:

```
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/
```

И в терминале введите:

**flutter pub get**

4. В файле **photo\_page.dart** в начало класса **\_PhotoPageState** добавьте список ваших изображений:



```
final List<String> _localPhotos = [
  'assets/images/photo1.jpg',
  'assets/images/photo2.jpg',
  'assets/images/photo3.jpg',
];
```

5. В методе `_fetchPhoto()` обновите блок `try/catch`:

```
try {
  // Имитируем сетевую задержку (чтобы был красивый loading)
  await Future.delayed(const Duration(seconds: 2));
  // Рандомно выбираем фото
  final randomIndex =
    DateTime.now().millisecondsSinceEpoch %
    _localPhotos.length;
  _imageUrl = _localPhotos[randomIndex];
} catch (e) {
  _errorMessage = 'Ошибка загрузки фото';
}
```

6. В методе `_buildContent()` поменяйте `Image.network` на `Image.asset` в блоке `if (_imageUrl != null)`:

```
if (_imageUrl != null) {
  return Expanded(
    child: Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      child: ClipRect(
        borderRadius: BorderRadius.circular(16),
        child: Image.asset(✓
```

### Самостоятельное задание. Оформление README.md

Оформите **README.md** в корне проекта `photo_of_the_day`

**Что должно быть в README:**

#### 1. Заголовок и описание

Название и одно-два предложения о проекте.

#### 2. Информация об авторе

Ваше имя, группа.

#### 3. Стек и версии

Flutter версия, Dart версия, платформа: Web (Chrome), пакет: http.

#### 4. Скриншот приложения

Используйте скриншот из папки `img/`.

## 5. Как запустить

### ## Запуск

1. ``git clone <url>``
2. ``cd photo_of_the_day``
3. ``flutter pub get``
4. ``flutter run -d chrome``

## 6. Что изучили

3–5 пунктов о новых концепциях.

## 7. Ответы на вопросы:

1. Что такое **Future<T>**? Чем отличается от обычного возвращаемого значения?
2. Что делает **await**? Блокирует ли он весь поток выполнения?
3. Зачем **setState()** вызывается дважды в **\_fetchPhoto()**?
4. Почему кнопке передаётся **\_fetchPhoto** без скобок, а не **\_fetchPhoto()**?
5. Чем **Image.network()** отличается от **Image.asset()**?

Сделайте финальный коммит:

**git add .**

**git commit -m "added README and completed labs"**

**git push**

### Итог — что изучили

| Концепция                        | Описание                                                              |
|----------------------------------|-----------------------------------------------------------------------|
| <b>Future&lt;T&gt;</b>           | Объект-обещание: вернёт значение типа T в будущем                     |
| <b>async</b>                     | Помечает функцию как асинхронную, разрешает использовать <b>await</b> |
| <b>await</b>                     | Приостанавливает функцию до завершения Future, не блокируя поток      |
| <b>try/catch</b>                 | Перехватывает ошибки в асинхронном коде                               |
| <b>http.get()</b>                | Отправляет GET-запрос и возвращает <b>Future&lt;Response&gt;</b>      |
| <b>jsonDecode()</b>              | Парсит JSON-строку в Map или List                                     |
| <b>CircularProgressIndicator</b> | Виджет-крутилка, индикатор загрузки                                   |
| <b>Image.network()</b>           | Загружает и отображает картинку по URL                                |
| <b>ChoiceChip</b>                | Виджет для выбора одного варианта из нескольких                       |